

AEGIS-Q: Evidence-First Hybrid Cryptographic Storage on Unified-Memory Edge Systems

Changjong Kim

changjong5238@seoultech.ac.kr

Seoul National University of Science and Technology
Seoul, Republic of Korea

Yongseok Son

sysganda@cau.ac.kr

Chung-Ang University
Seoul, Republic of Korea

Ehan Sohn

ehanjsohn@seoultech.ac.kr

Seoul National University of Science and Technology
Seoul, Republic of Korea

Sunggon Kim

sunggonkim@seoultech.ac.kr

Seoul National University of Science and Technology
Seoul, Republic of Korea

Abstract

Unified-memory edge devices co-locate foreground inference, local databases, and encrypted state on shared DRAM/s-storage. A secure-storage runtime must place cryptographic work without disrupting latency tenants, publish durable authenticated state without nonce reuse, and bind freshness outside replayable backing directories. AEGIS-Q explores this point as a FUSE prototype with one authenticated block format, a password-derived mount-key boundary, a CPU-first AES-GCM data plane, an optional batched GPU maintenance lane/rekey fallback, and a fail-closed external-anchor TPM replay-after-advance path. On a Jetson AGX Thor Developer Kit, retained mounted artifacts show the intended spine: under secure-storage pressure, foreground SQLite p99 rises from 6.4 ms to 13.8 ms and incurs a deadline miss; AEGIS-Q’s policy recovers p99 to 8.8 ms with zero misses while retaining 2.7 MB/s of background progress. The same evidence set exercises remount/tamper checks, generation/fault matrices, rekey fallback, TPM replay-after-advance, app-crash timing, and telemetry-to-FUSE throttling. The result is an evidence-first secure edge-storage prototype with explicit scope boundaries, not a claim of deployed-filesystem superiority.

Keywords

Post-Quantum Cryptography, Edge Computing, Secure Storage, Unified Memory, TPM, FUSE

1 Introduction

Edge devices increasingly run foreground AI inference, local databases, and encrypted storage on a shared DRAM and storage fabric. This deployment changes the usual secure-storage question. Confidentiality and integrity remain mandatory, but a storage daemon that treats the integrated GPU as a free accelerator can contend with the very foreground workload it is supposed to protect. The resulting problem is how to build a durable authenticated storage format whose work-placement policy is explicit, measurable, and safe under unified-memory assumptions.

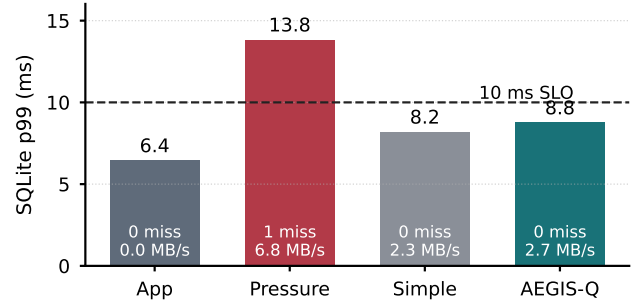


Figure 1: SQLite pressure result for RQ3; labels show deadline misses and background throughput.

Table 1: Capability comparison that defines AEGIS-Q’s design gap. “Policy” means storage-visible accelerator/QoS/freshness policy, not implementation maturity.

System	File/blk enc.	Fresh root	UMA policy	Boundary
Plaintext	✗	✗	✗	Lowerfs baseline only
gocryptfs	✓	✗	✗	User-space encryption baseline
fsencrypt	✓	✗	✗	In-kernel file encryption
dm-crypt	✓	✗	✗	Block-device encryption
fs-verity/dm-integrity	△	✗	✗	Read-only file or block integrity
TPM/TEE-backed designs	△	✓	✗	Hardware trust boundary
GPU-storage systems	△	✗	△	Accel. path focus
AEGIS-Q	✓	△	✓	FUSE prototype with scoped evidence

Post-quantum key establishment is relevant to long-lived edge data, but it does not replace symmetric storage encryption. ML-KEM establishes shared secrets, whereas AES-GCM remains the appropriate mechanism for bulk data records [6]. SHA-256 can bind committed-prefix anchor material to persisted metadata [5]. The systems challenge is therefore to compose ordinary storage invariants—a key hierarchy that survives remount, nonce/generation state that remains safe across crashes, ciphertext durability before metadata publication, and fail-closed recovery—with accelerator placement and shared-resource control. Table 1

positions that gap against plaintext, gocryptfs, fscrypt, dm-crypt, fs-verity/dm-integrity, TPM/TEE-backed designs, and GPU-storage systems [2, 3, 8, 10].

Figure 1 shows the retained first-page pressure result. Mounted secure-storage pressure raises foreground SQLite p99 from 6.4 ms to 13.8 ms and introduces one 10 ms deadline miss. AEGIS-Q marks foreground files latency-sensitive, throttles only elastic background writes, and recovers to 8.8 ms p99 with zero misses while retaining 2.7 MB/s of background progress. This result motivates the paper’s spine: policy must be visible at the secure-storage layer, but persistent format correctness must remain independent of the accelerator.

AEGIS-Q explores this gap as a UMA-aware secure-storage runtime. Its thesis is that secure edge storage should expose one durable authenticated format while separating three operational planes: a CPU-first data plane for latency-sensitive AES-GCM writes, an elastic GPU lane for large independent cryptographic batches, and an external-anchor path that can fail closed when replayed state no longer matches protected freshness state. The claim is stronger than a microbenchmark-only story because it ties placement decisions to format invariants and recovery behavior.

The key design insight is that accelerator placement must be subordinate to storage correctness. AEGIS-Q never lets the GPU define the persistent record format. Data records are AES-GCM blocks bound to file identifier, logical block, generation, and plaintext length. Metadata publication follows a data/journal/checkpoint order. Optional GPU work either produces format-compatible results or falls back to the CPU path. Jobs without producer-supplied slack are routed conservatively. This makes the accelerator an execution choice, not a new security boundary.

This paper makes four contributions.

- (1) **C1: Secure-storage format and placement boundary.** AEGIS-Q implements authenticated per-file envelopes, generation-bound AEAD context, durable mapping publication, and explicit CPU/GPU placement rules. Design/evaluation: Section 3.2 and Section 6.3.
- (2) **C2: Scoped placement and workflow evidence on Thor.** Retained diagnostic primitive runs and a mounted envelope-refresh workflow explain the CPU-first data lane and optional batched GPU lane. Design/evaluation: Section 3.3 and Section 6.2.
- (3) **C3: Recovery/freshness evidence with failure labels.** The repository retains clean-remount, tamper-rejection, file-anchor negative-control, TPM replay-after-advance, SQLite/dbm.dumb stale replay, and SQLite syscall-exact app-crash evidence. Design/evaluation: Section 3.4 and Section 6.3.
- (4) **C4: Telemetry/QoS evidence.** AEGIS-Q includes deterministic admission traces, same-run tegrastats/CUPTI-to-mounted-FUSE throttling, and the mounted SQLite recovery bundle in Figure 1. Design/evaluation: Section 3.3 and Section 6.4.

The paper is intentionally conservative about unsupported mechanisms. It does not claim verified 0.DIRECT NVMe-to-UVM DMA, an `io_uring`/eBPF completion bypass, GPU constant-time execution, persistent PCR-bound freshness, true power-loss crash certification, or portability beyond the tested stack. These exclusions are not a substitute for contributions; they define the boundary within which the measured system should be read.

The rest of the paper follows the structure of a systems design argument. Section 2 states the storage and UMA assumptions, Section 3 presents the format and placement design, Section 4 records implementation choices that determine the boundary, Section 5 states the security and recovery model, Section 6 evaluates the retained claims, and Sections 7–9 discuss implications, related work, and limits.

2 Background and Scope

2.1 Secure storage primitives

Secure-storage systems choose a protection boundary as well as a cipher. Kernel filesystem encryption, user-space encrypted filesystems, block encryption, and TEEs occupy different boundaries and trust assumptions [2, 3, 8, 10]. For example, fscrypt [3] and OP-TEE [8] operate at substantially different boundaries. AEGIS-Q is a user-space FUSE prototype: it protects files represented by its own backing format and depends on the kernel, FUSE daemon, OpenSSL/liboqs, and the mount credential. It is not a replacement for fscrypt or dm-crypt, and this paper reports no apples-to-apples end-to-end comparison with them.

The cryptographic roles are deliberately distinct. AES-GCM encrypts and authenticates each data block; SHA-256 constructs the committed-prefix root used by the anchor helper; and ML-KEM-768 is a key-establishment primitive rather than a bulk-data primitive [5, 6]. We use ML-KEM only for the optional batched key-plane experiment. The production FUSE data path derives a random file data-encryption key (DEK), wraps it under a mount-derived key, and uses that DEK for AES-GCM. This avoids the atypical and expensive design of performing a KEM operation for every block.

2.2 Why another user-space prototype?

The value of AEGIS-Q is not that user-space encryption is inherently preferable to kernel encryption. In fact, kernel-integrated systems have a clear maturity advantage. AEGIS-Q is also not merely gocryptfs/fscrypt plus CUDA/TPM scripts: the research question is whether one mounted runtime can keep authenticated-block publication, storage-visible QoS/admission, optional batched GPU key-plane work, and external freshness checks under one evidence contract. In that setting, the storage layer exposes policy decisions that conventional encryption layers usually hide. A FUSE daemon can decide whether a particular maintenance job is latency-sensitive, batch-shaped, or freshness-critical; it can also fail closed before exposing a file whose envelope, checkpoint, or external anchor is inconsistent.

The capability matrix on the first page is not a performance ranking. It explains why the paper evaluates AEGIS-Q as a placement and recovery prototype rather than as a replacement for mature deployed filesystems.

2.3 Unified memory is not a DMA contract

On a unified-memory SoC, CPU and GPU share DRAM capacity and bandwidth; Jetson-class systems are a representative deployment setting [7]. CUDA managed memory makes an allocation accessible to CPU and GPU code, and prefetch is a performance hint; neither API is a permission bit nor a storage-DMA registration protocol. The present implementation allocates managed buffers only inside the CUDA executor, prefetches them around a GPU operation, and synchronizes the stream before CPU reuse. The backing files are accessed with ordinary `pread/pwrite`. It does *not* call `cudaHostRegister`, issue `0_DIRECT` into managed memory, or rely on `cudaStreamAttachMemAsync` as mutual exclusion.

This scope matters because systems work must separate an API’s correctness semantics from an optimization hypothesis. A future direct-I/O path would need an explicit driver/version matrix, pinning and IOMMU evidence, a fault-injection campaign under memory pressure, and a protocol that excludes DMA, CPU, and GPU concurrent access. None of those conditions is established here. The absence of that path is a design constraint, not an unreported optimization.

This distinction also affects how to read the retained auxiliary diagnostics. A successful same-buffer checksum through a mapped host pointer shows that a particular buffer was visible to a CUDA kernel after a storage read. It does not establish that the production FUSE path can safely register every file-cache page, that the NVMe controller can DMA into managed memory without migration hazards, or that page-fault and coherence counters are available on the deployed driver stack. AEGIS-Q therefore treats managed memory as an executor-local convenience and treats storage I/O as POSIX sidecar I/O unless a future driver-level campaign proves more.

2.4 Why work placement is asymmetric

GPU acceleration is useful only when it amortizes launch, placement, and synchronization costs. Prior out-of-core systems such as FlashNeuron and Fastensor focus on moving large learning workloads through storage hierarchies [1, 11]; their complementary staging perspectives are also relevant to the batch boundary considered here [1, 11]. GPU-oriented work such as `fsencrypt-GPU` and FlashNeuron further motivates separating batch placement from the storage format [1, 13]. They do not validate this FUSE format or its durability protocol. Likewise, storage-oriented systems including SnuQS and InfScaler motivate explicit staging but do not make an NVMe/UVM correctness claim for this prototype [4, 9]. The evaluation therefore asks a smaller question: for each

measured primitive, does CPU or GPU offer the better observed throughput on the tested platform? Section 6 reports the answer and retains the raw logs.

The asymmetric answer is useful for system design. Bulk data encryption is on the critical path of small writes, SQLite commits, and foreground reads. Those operations are sensitive to launch latency, staging overhead, and CPU/GPU synchronization. By contrast, key rotation, batch encapsulation, and committed-prefix hash maintenance can produce many independent jobs whose latency can be amortized and whose execution can be deferred when the foreground workload is under pressure. AEGIS-Q’s scheduler therefore does not ask whether the GPU is faster in isolation. It asks whether a job is large enough, independent enough, and slack-tolerant enough to justify entering the elastic lane.

2.5 Design requirements

The resulting system requirements are:

- (1) **One storage format.** CPU and GPU execution must produce the same authenticated record format, so fallback does not change recovery semantics.
- (2) **Publication before exposure.** A write is not visible merely because ciphertext bytes exist; a validated journal mapping and checkpoint determine exposure.
- (3) **No optimistic accelerator semantics.** CUDA managed memory and prefetch are local executor mechanisms until storage-DMA behavior is separately proven.
- (4) **Fail-closed freshness.** A file-backed witness is an integrity check, not a freshness root; external anchors must be evaluated separately.
- (5) **Admission before acceleration.** A job with no explicit slack or stale telemetry must take the CPU path even if the GPU is present.

These requirements shape the rest of the design. They also explain the paper’s evaluation style: negative controls and scoped failures are part of the evidence because they prevent the prototype from inheriting guarantees from mechanisms it does not actually implement.

3 AEGIS-Q Design

3.1 Planes and trusted state

Figure 2 separates the persistent secure-storage path from optional accelerator work. The data plane is an AES-256-GCM block format. The key-plane workflow contains optional batched ML-KEM envelope refresh for open files; it does not encrypt ordinary file blocks. The integrity helper computes SHA-256/Merkle material for parity tests and for the committed-prefix anchor root; AEGIS-Q does not persist a per-file content Merkle tree. The freshness plane publishes an authenticated checkpoint and can target a pre-provisioned TPM NV index; this revision evaluates only fail-closed replay-after-advance behavior for that hardware path.

The design is organized around seven invariants that came directly from the threat model and the review concerns: authenticated key exposure, nonce/generation safety, durable publication ordering, checkpoint exposure, external freshness, unsupported-mode handling, and controller fallback. Table 2 is the audit boundary for those invariants. Each row names the implementation locus, the retained evidence that exercises the invariant, and the narrower paper claim when the current artifact does not close the full production-grade version.

Mount-key derivation uses PBKDF2 with HMAC-SHA256. New files receive random 256-bit DEKs and file identifiers; each DEK is masked under mount-key/file-ID material and HMAC-authenticated before open accepts it. The file identifier, generation, logical block number, and plaintext length are AES-GCM associated data, preventing valid records from moving to different logical positions.

The mount key is password-derived and never hardware-released; there is no hardware-backed credential release path for the mount key. The rekey is open-file DEK refresh rather than deployed credential rotation: HMAC envelope rewrap for open files; runtime key epoch is in-memory bookkeeping, not a persistent anti-rollback journal. There is no transactional rewrap recovery claim, and password-derived envelope alone is not rollback resistance; stale old envelopes fail only when generation/checkpoint/external-anchor evidence rejects them.

3.2 Durable encrypted-block publication

Writes are coalesced only when they are contiguous. For each affected 4 KiB logical block, AEGIS-Q reads the current authenticated block if necessary, applies the modification, assigns a strictly increasing generation, and derives the GCM nonce from (*fileID*, *block*, *generation*). The commit order is:

- (1) write ciphertext records to `.pqcddata` and call `fdatasync`;
- (2) append generation-to-offset mappings and tags to `.pqcmeta`, then call `fdatasync`;
- (3) update the logical-size xattr; authenticate and write the checkpoint.

Figure 3 makes the publication state machine explicit. It is intentionally not a proof diagram: the transition table records which edges are exercised by retained artifacts and which crash boundaries remain outside the evaluated envelope.

The journal is the publication point: a ciphertext tail with no durable mapping is unreachable after recovery. Conversely, a mapping is written only after its ciphertext range has crossed the data durability barrier. `fsync` flushes the coalescing buffer, waits for local pending work, and calls `fdatasync` on both sidecars before returning. This is an implementation argument for the tested format, not a proof of full POSIX behavior under every FUSE cache mode or an SQLite certification.

The generation rule is intentionally simple. A generation value is consumed when a new record is constructed for a logical block; recovery accepts only records reachable through a validated mapping and checkpoint. Reusing an older generation for a new plaintext would be a correctness bug because the GCM nonce is derived from file identifier, block, and generation. The retained generation matrix exercises partial update, clean remount, torn journal tail, stale snapshot replay, and an adversarial older-generation append after a newer mapping; the daemon cutpoint matrix adds selected one-block SIGKILL timing. The paper still does not claim physical power-loss or broad crash-consistency certification.

Unsupported POSIX modes are part of the design boundary. The retained audit confirms the implemented contract: ordinary read/write/`fsync` and narrow disjoint writers are supported; default encrypted-file `mmap`, rename, and directory `fsync` are rejected; internal checkpoint/envelope xattrs are hidden; and FUSE writeback-cache/direct-IO `mmap` capabilities are not requested. Arbitrary conflicting concurrency and directory-entry durability remain outside the claim.

3.3 Managed-memory boundary and scheduler

Table 3 summarizes the claims that the source code actually makes: accelerator placement must be subordinate to storage correctness, CPU/GPU ownership is protected by the FUSE per-file lock and stream completion, and CUDA prefetch is used only to influence locality. No table entry is a statement about NVMe page pinning, cache coherence of a third-party DMA engine, or CUDA attach-mode permissions.

The placement function balances workloads between the ARM CPUs and the Jetson GPU. The current revision does not expose a verified `O_DIRECT` storage path into GPU memory. The CUDA executor instead uses managed allocation, explicit prefetch, and stream synchronization as locality and ordering controls.

QoS follows the SQLite contract in Figure 1: secure-storage pressure raises AEGIS-Q p99 from 6.4 ms to 13.8 ms; classification recovers 8.8 ms, zero deadline misses, and 2.7 MB/s background progress, not foreground-inference p99 or completion-bypass control.

The producer-facing interface is deliberately conservative. A job can enter the elastic lane only when its size and batch shape make GPU execution plausible and the producer supplies nonzero relative slack in nanoseconds. The trace records `CLOCK_REALTIME` timestamps for correlation; batch age, deadline, and slack age are `CLOCK_MONOTONIC`-relative and need no cross-process clock synchronization. Missing, zero, or stale slack falls back to CPU with the exact deferral reason. The mounted path also exposes `user.pqc_qos_class`: latency files are traced but never daemon-slept, elastic files are throttle-eligible, and SQLite sidecars inherit the base-file class.

The evaluated key-plane workflow applies this rule by batching open-file descriptors, encapsulating on the selected

Table 2: Formal storage-protocol invariants, implementation loci, retained evidence, and claim boundaries.

Invariant	Implementation locus	Retained evidence	Claim boundary
Envelope authentication before DEK use	<code>metadata.load()</code> verifies the HMAC-protected envelope before <code>pqc.open()</code> installs file state	Clean-remount regression and <code>user.pqc.metadata</code> tamper rejection returning <code>EKEYREJECTED</code>	Remountable authenticated envelope; no hardware-bound credential release
AEAD nonce uniqueness	<code>derive_block_nonce()</code> , <code>build_block_aad()</code> , and <code>do_flush_wbuf_locked()</code> bind file identifier, block, generation, and length	Generation fault matrix, self-test replay regression, clean-remount validation, and daemon cutpoint matrix	Generation-bound record format; physical power loss and broad workload certification remain open
Data-before-mapping publication	<code>do_flush_wbuf_locked()</code> writes <code>.pqcddata</code> , calls <code>fdatasync</code> , appends <code>.pqcmeta</code> , then syncs the journal	Journal self-test, daemon cutpoint matrix, and SQLite selected-boundary campaign	D/J ordering evidence; no physical power-loss or drive-cache certification
Checkpoint exposure rule	<code>logical_size.store()</code> , <code>checkpoint.store()</code> , <code>checkpoint.load()</code> , and <code>remount.ctx.set()</code> expose size/checkpoint state only after journal publication	Checkpoint self-test, clean remount, and stale-replay fail-closed artifacts	Authenticated checkpoint exposure; lower-filesystem xattr/directory ordering is delegated
External-anchor freshness condition	<code>checkpoint.store()</code> propagates hardware-anchor failures and <code>checkpoint.load()</code> rejects mismatched anchor state	TPM NV replay-after-advance run fails closed; file-anchor replay is retained as a negative control	Replay-after-advance freshness only; no persistent PCR-bound lifecycle claim
Unsupported-mode boundary	<code>pqc.open()</code> defaults to FUSE <code>direct.io</code> ; <code>rename</code> and directory <code>fsync</code> return <code>ENOTSUP</code> ; internal xattrs are hidden	POSIX scope audit plus current remount/SQLite selected-boundary tests	Ordinary read/write/ <code>fsync</code> and narrow disjoint writers only; shared <code>mmap</code> and directory-entry durability are not claimed
Controller safety fallback	<code>pqc.block.job.choose_target()</code> and <code>pqc.admit()</code> default to CPU on missing slack, stale telemetry, size, deadline, or queue-pressure constraints	Scheduler self-test, admission sweeps, and same-run CUPTI/tegrastats-to-mounted-FUSE throttle traces	Implemented conservative fallback; no foreground inference p99 recovery claim

Table 3: Memory-path claims and their hardware-verified status.

Mechanism	Status	Claim retained
<code>cudaMallocManaged</code> + prefetch	Verified	Managed allocation and explicit device/host prefetch complete.
<code>posix.memalign</code> + <code>cudaHostRegister</code>	Prototype-only	Host-memory pinning and executor-local access checks; not a verified storage-DMA path.
<code>pread/pwrite</code> sidecars	Verified	Backing ciphertext and journal use ordinary POSIX I/O.
eBPF tracepoint notification	Not claimed	No final eBPF notification path is part of the verified implementation.
QoS telemetry	SQLite recovery only	Workload/CUPTI traces can drive mounted-FUSE throttling; no foreground AI p99 recovery claim.

executor, installing 32-byte refreshed DEK material, and rewriting the HMAC-authenticated envelope. It is not a persistent KEM hierarchy or hardware-backed credential lifecycle.

3.4 Freshness boundary

The file anchor is an authenticated on-disk record. Because an attacker able to restore the backing directory can restore that record as well, it is explicitly *not* rollback protection. The crash/replay harness treats this behavior as a negative control. The optional hardware backend refuses to mount when its TPM NV index was not provisioned by an administrator; it never creates persistent TPM state implicitly. The retained hardware evidence is narrower: replay after the anchor advances fails closed. A stronger rollback-resistance claim requires a separately protected TPM policy, defined NV authorization, a recovery protocol, and power-loss results for that exact configuration.

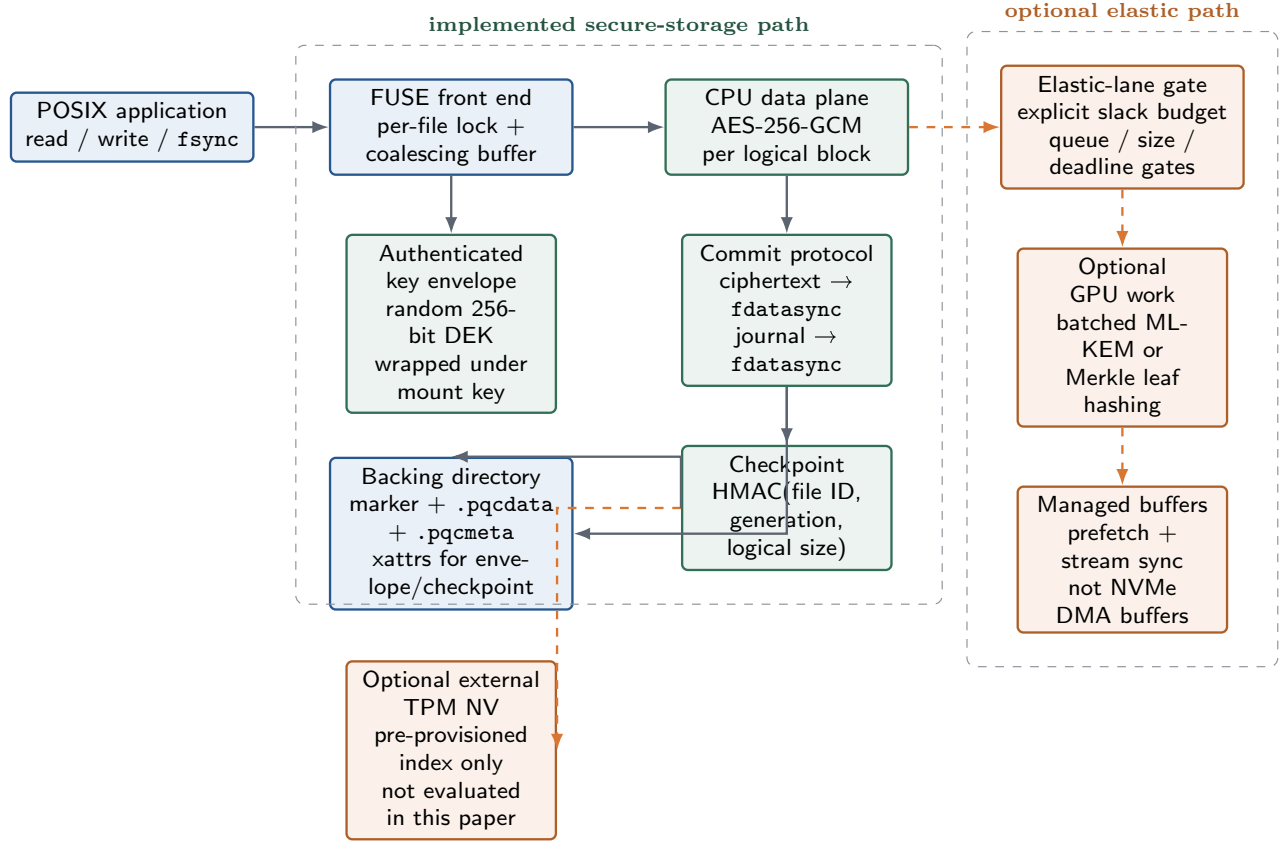
4 Implementation

AEGIS-Q is implemented in C/CUDA on FUSE3. The secure-storage path is in `pqc.fuse.c`; the anchor format and Merkle helper are in `pqc.anchor.c`; CUDA AES-GCM, ML-KEM, and integrity executors are separate translation units. The build enables `-Wall -Wextra -Werror` for C/C++ and builds CUDA targets only when the required cuPQC libraries are present. The reported build used liboqs 0.15.0, FUSE3 3.14.0, OpenSSL, CUDA 13.0.48, and cuPQC on the platform in Section 6.

The implementation makes several choices that are easy to obscure in a high-level diagram; Table 4 summarizes the boundary for each one. First, GPU AES-GCM is format-compatible with the CPU implementation, but failure in the CUDA executor falls back to CPU rather than changing a record format. Second, the GPU ML-KEM batch executor allocates device buffers internally; it is not an in-place filesystem data path. Third, the integrity executor has direct parity tests against OpenSSL for leaf hashes, non-power-of-two trees, and the empty-tree case. These tests check functional equality for the helper used by the committed-prefix anchor root; they do not mean the filesystem persists a per-file content Merkle tree or provides side-channel protection.

The key-envelope repair is central to remount correctness. An earlier metadata design generated and discarded a per-file ML-KEM secret key, making later decapsulation impossible. The revised path generates a random DEK, wraps it under the mount key, HMAC-authenticates the envelope, and rejects failed authentication before ciphertext is read. ML-KEM is only optional batched maintenance: the rekey worker can refresh open-file DEK material and persist new HMAC envelopes, but the format does not pretend that a KEM benchmark is a persistent per-file key hierarchy.

The older `pqc.file_key.*` epoch/grace helper remains source-only and is not linked into `pqc.fuse`; evaluated



The implementation uses ordinary POSIX `pread/pwrite`; it does not claim `O_DIRECT` into managed memory, `io_uring` completion bypass, or eBPF-triggered CUDA work.

Figure 2: Implemented AEGIS-Q control and persistence path. Solid arrows are the FUSE format used by the tests; orange dashed arrows denote optional accelerator or externally provisioned hardware paths. The final revision does not claim verified `O_DIRECT` UMA pinning, a completed eBPF completion path, or a closed-loop hardware-QoS controller.

epochs are open-file refresh bookkeeping, not stale-handle, rollback, or TPM/PCR-bound freshness proof.

The implementation also fails closed in two places. A scheduler job with no producer-supplied AI slack is routed to CPU. A requested TPM backend with no pre-provisioned NV index causes initialization to fail rather than creating a persistent index with undocumented ownership. These are conservative operational choices; neither substitutes for a complete inference-telemetry integration or TPM policy design.

4.1 Persistent-object layout

Each protected file is represented by ordinary lower-filesystem state rather than an in-kernel extent map. The data sidecar stores encrypted records; the metadata sidecar stores logical-block to record mappings and authentication material; xattrs store compact metadata such as logical size and the authenticated envelope. This layout is intentionally

inspectable for fault-injection experiments: a harness can copy or corrupt a backing directory, restore an older sidecar, or mutate an xattr without requiring a custom block device.

The layout also creates concrete assumptions. AEGIS-Q relies on the lower filesystem to honor `fdasync` on the data and metadata sidecars and to preserve xattr updates according to its documented semantics. It does not claim that every directory-entry or rename edge case has been exercised. The paper therefore separates the implementation invariant—data is synced before a mapping is published—from the stronger certification claim that every possible lower-filesystem crash interleaving has been tested.

4.2 Concurrency and unsupported modes

The evaluated path uses ordinary FUSE read, write, flush, fsync, open, and release operations. Per-file state is serialized through the daemon’s state management before records are published. The coalescing buffer is flushed before sync

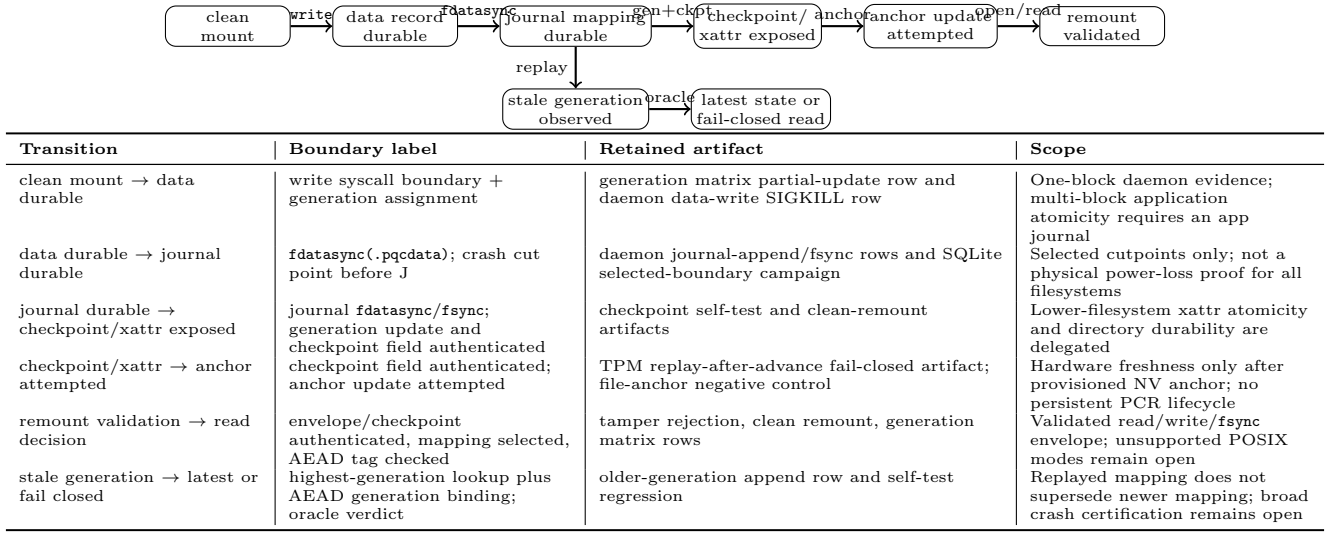


Figure 3: D/J/C/xattr publication state machine and transition audit. D is the durable ciphertext sidecar state, J is the journal mapping state, and C is the authenticated checkpoint/logical-size exposure state.

Table 4: Implementation boundaries that are easy to mis-read without an explicit summary.

Mechanism	Current implementation	Boundary
CUDA storage path	Managed allocation with explicit prefetch and stream synchronization	Not a verified NVMe-to-UVM storage-DMA path
CUDA failure policy	CUDA executor falls back to CPU on failure	Does not change the on-disk record format
Anchor backend	File witness plus optional hardware NV index backend	Hardware path requires administrator pre-provisioning
Scheduler fallback	No-slack jobs route to CPU	No optimistic GPU admission without producer-supplied slack

returns, and pending block jobs are drained before the checkpoint is considered visible. This is sufficient for the retained remount and SQLite selected-boundary experiments.

The implementation is not presented as a full POSIX conformance layer. In particular, the paper does not claim validated shared `mmap` semantics, all possible concurrent-writer races, cross-process byte-range lock compatibility, or arbitrary rename/directory-fsync crash behavior. Those modes are common sources of filesystem bugs, and the review correctly identified them as missing evidence. Treating them as

unsupported or unvalidated is preferable to silently inheriting a POSIX claim from FUSE.

4.3 Telemetry plumbing

The admission logic is implemented as ordinary C state rather than a separate daemon that invents scheduling results. It records input queue depth, batch size, slack, telemetry freshness, and route decision. The mounted-FUSE throttle path reads a telemetry file and applies hysteresis before sleeping inside the write-flush path. The tegrastats and CUPTI harnesses differ only in how they populate that telemetry input. This shared path is the reason the evaluation can claim same-run telemetry-to-FUSE wiring. It is also why the paper still requires a separate foreground-inference experiment before making an AI QoS-restoration claim.

5 Security and Recovery Scope

5.1 Threat model

AEGIS-Q considers an attacker who can read, copy, modify, and replay the backing directory but cannot obtain the mount credential or compromise the running kernel, FUSE daemon, CUDA driver, or cryptographic libraries. Under that scope, AES-GCM protects the confidentiality and authenticity of published block records, while the HMAC-protected key envelope and checkpoint detect unauthenticated metadata changes. The adversary can still deny service, delete records, replay a complete file-backed snapshot, or exploit vulnerabilities outside the FUSE process. We make no claim against a privileged local attacker that can read process memory, substitute the CUDA driver, alter the mount executable, or use GPU microarchitectural side channels.

Table 5: Threat classes and retained mechanism boundaries.

Threat	Mechanism	Boundary
Backing-store read	AES-GCM ciphertext with per-file DEK	Assumes mount credential remains secret
Backing-store tamper	AEAD tags, HMAC envelope, HMAC checkpoint	Denial of service remains possible
Whole-directory replay	Optional external TPM anchor can fail closed after advance	File-only witness is replayable; full rollback resistance is not claimed
Crash during update	Ordered publication plus selected daemon/app cutpoint tests	Physical power-loss and drive-cache certification remain open
GPU side channel	No protection mechanism claimed	Integrity parity is not constant-time evidence
Privileged local attacker	Out of scope	Kernel/driver/FUSE process are trusted

The accepted encrypted state consists of a validated DEK envelope, a complete journal mapping, and ciphertext records referenced by that mapping. A tag failure causes a read to fail. A malformed or unauthenticated envelope/checkpoint causes open to fail. Table 5 states the threat classes these mechanisms address and the boundaries left outside the paper. These behaviors are necessary for a safe FUSE format, but they are not a formal proof of crash consistency. In particular, xattr atomicity and directory durability are delegated to the underlying filesystem.

5.2 Freshness is conditional on an external anchor

The source tree contains two anchor modes. The file mode is only an integrity witness; copying the directory copies the witness. The corrected crash/replay run therefore reports four of four file-backed snapshot restores as *rollback visible*, not as successful protection. This negative result is important: an HMAC alone does not create freshness when its entire state is replayable.

The hardware mode stores an authenticated prefix record in an owner-authorized TPM NV index: administrators create and rotate the index outside the daemon, and AEGIS-Q reads or writes only that record. PCR binding is a transient probe rather than a persistent filesystem policy; a software update or PCR change therefore requires explicit reprovisioning/resealing. Backup or migration cannot copy only the backing directory; it needs a fresh authorized anchor bootstrap. Lost mount credentials are unrecoverable. Missing index, authorization failure, TPM I/O failure, digest mismatch, or a TPM sequence ahead of local state fail closed; a TPM anchor behind local state is treated as a lagging witness and must be refreshed rather than as disk rollback. The evaluated claim is only replay-after-advance fail-closed behavior, including the SQLite same-store campaign in Section 6.3; it does not assert that an NV index is a signer, report invented TPM latency, or claim persistent PCR binding.

5.3 Side channels and application semantics

GPU SIMT execution, cache behavior, memory coalescing, occupancy, and contention can all leak information. The repository contained generators that produced simulated timing and TVLA traces. Those files are excluded from this paper. The retained GPU integrity test checks output parity with OpenSSL; it says nothing about constant-time execution, physical power/EM leakage, or cross-tenant GPU channels.

Likewise, the clean-remount test validates one encrypted 128 KiB round trip through the final binary. It does not certify SQLite WAL, `mmap`, locking, crash recovery across every cut point, or the complete POSIX contract. We retain the following labels to make the scope explicit: the prior SQLite case study (Section 5.4) and edge/cloud comparison (Section 5.5) are not claimed by this revision.

Application semantics are separated from storage-record integrity. AEGIS-Q can reject a corrupted block tag, but it cannot infer whether an application-level transaction was acknowledged unless the experiment supplies an external oracle. The daemon power-fault matrix adds final-binary SIGKILL at selected one-block storage boundaries, while the SQLite WAL/FULL and DELETE/EXTRA rows supply application commit oracles. A complete database certification would need broader workloads, multiple cache modes, physical power loss, kernel crash, and drive-cache timing.

5.4 What the current tests establish

The final validation runs a FUSE self-test, clean encrypted remount, a managed-memory smoke test, and GPU integrity parity checks. The round-trip test confirms that after a clean unmount/remount, the original 128 KiB plaintext is recovered and the corresponding `.pqcddata` sidecar does not contain the whole plaintext byte sequence. This is evidence of the repaired key-envelope path, not a statistical security proof.

5.5 Deployment boundary

AEGIS-Q is a prototype for local edge storage. It makes no edge-versus-cloud cost, latency, or privacy quantification. The appropriate comparison is deployment-specific and requires a separately measured network/service configuration. This restriction avoids turning a local microbenchmark into a claim about cloud databases or remote key-management systems.

6 Evaluation

6.1 Methodology and provenance

The evaluation answers five research questions:

- **RQ1: Correctness.** Does the implemented authenticated format preserve nonce/generation, publication, and metadata-authentication invariants under retained remount, fault, and tamper campaigns?

- **RQ2: Mode-aligned baselines.** What cost does AEGIS-Q impose relative to retained plaintext and gocryptfs rows under the frozen filesystem contract?
- **RQ3: QoS/hero result.** Does AEGIS-Q recover foreground SQLite latency under mounted secure-storage pressure relative to unthrottled pressure and a simple controller?
- **RQ4: Freshness and recovery.** What oracle-classified recovery and TPM-freshness outcomes are supported by the retained fault campaigns?
- **RQ5: Ablation and sensitivity.** Which mechanisms account for the measured placement, controller, key-plane, and anchor behavior, and how sensitive is the SQLite controller?

The design/evaluation contract is one closure per main mechanism: authenticated block format closes through the generation/tamper evidence in RQ1; D/J/C publication closes through oracle-labeled daemon/app cut points in RQ4; CPU/GPU placement closes through the data-plane and key-plane measurements in RQ2/RQ5; TPM freshness closes through the hardware-freshness matrix in RQ4; the QoS controller closes through SQLite recovery and sensitivity traces in RQ3/RQ5; and the optional ML-KEM lane closes through the mounted open-file rekey workflow in RQ5.

Primitive-placement numbers come from the verified microbenchmark harness; the mounted rekey result comes from the key-plane workflow harness. Both capture provenance and refuse to manufacture unsupported configurations. The completeness gate keeps baseline cost, mounted pressure, workload diversity, placement breakdown, sensitivity, stability limits, and the SQLite case study retained while leaving missing kernel rows and statistical generalization to Discussion.

The platform was an NVIDIA Jetson AGX Thor Developer Kit with 14 ARM CPU cores, a 1 TB WD PC SN5000S NVMe device, Linux 6.8.12-tegra, CUDA 13.0.48, liboqs 0.15.0, FUSE3 3.14.0, and the cuPQC libraries found by CMake. The result is not an Orin Nano result and must not be generalized to other Jetson generations. The current evidence has three tiers: Figure 4 is a diagnostic placement result with three repetitions and observed-range whiskers, not confidence intervals; the key-plane and SQLite bundles are single retained workflow artifacts, not statistical confidence claims; future headline comparisons require warmup, five repetitions, bootstrap 95% confidence intervals, outlier policy, fixed governor/clock/power capture, thermal logs, background-process control, cache policy, and fail-closed handling for missing logs or failed modes.

Table 6 states what each experiment can and cannot establish. The v2 filesystem contract fixes 4 KiB 70/30 random read/write, `psync`, per-write `fdatasync`, warm/cold cache policy, queue depth 1, one client, a sparse-prepared 1 GiB file, fixed mounts/platform, five repetitions, and bootstrap

Table 6: Evaluation scope by research question.

RQ / exp.	Establishes	Does not establish
RQ1 FUSE	Format functions after clean remount	Full SQLite/crash certification
RQ1 hash	CUDA and OpenSSL digests agree	Side-channel resistance
RQ2 Frozen contract	Plaintext, gocryptfs, and AEGIS-Q warm rows	fsencrypt/dm-crypt and cold rows
RQ2/RQ5 UMA	Managed allocation and host/device prefetch complete	Verified storage-DMA or O_DIRECT-to-UVM
RQ2/RQ5 eBPF	<code>nvme.complete_rq</code> histogram retained	Mounted eBPF/io_uring path
RQ3 daemon	Scheduler prototype and telemetry scripts exist	End-to-end AI p99 controller
RQ3 stress	4-writer tail-latency probe under pressure	End-to-end AI workload
RQ3 TensorRT	Interference traces and samples are retained	Closed-loop QoS controller
RQ4 file anchor	File snapshot rollback remains visible	TPM-anchor freshness

95% confidence intervals. The plaintext warm-cache row reports 32.88 MiB/s median throughput and 0.165 ms conservative p99 (95% CIs 30.16–36.96 and 0.157–0.167); gocryptfs reports 21.72 MiB/s and 0.060 ms (21.19–21.86 and 0.0596–0.0622); the AEGIS-Q warm-cache row reports 0.359 MiB/s and 11.21 ms (0.286–0.547 and 7.70–13.70). The mechanism-ablation manifest uses these rows to gate the format gap and links the controller, key-plane, and anchor variants to the RQ5 evidence below; broader kernel-encryption rows are discussed in Section 7.3.

6.2 CPU for bulk data, GPU for batched ML-KEM

Figure 4 reports the diagnostic placement result. For a 16 MiB AES-256-GCM input, CPU/OpenSSL reaches 1.61 GB/s (observed 1.59–1.63 GB/s), whereas the managed-buffer GPU path reaches 0.172 GB/s (0.172–0.172 GB/s). The GPU result includes managed-buffer staging and stream synchronization. Smaller requests amplify the same effect. This is why AEGIS-Q’s data plane is CPU-first; a GPU label alone is not a performance result.

For a batch of 4,096 ML-KEM-768 key generations, liboqs on CPU reaches 64.8 K operations/s (64.3–64.9 K), while cuPQC reaches 1.50 M operations/s (1.49–1.51 M), a 23.2× ratio of medians. The comparison is a primitive microbenchmark on one platform, not a claim that a FUSE write becomes 23.2× faster. It supports the narrower policy of reserving GPU work for sufficiently large, independent key-plane batches.

The retained mounted ML-KEM-768 key-plane workflow opens 1,024 encrypted files and refreshes their HMAC-authenticated envelopes. CPU-only refresh takes 24.399 ms (41,969.6 files/s). With explicit slack, admission selects the GPU and finishes in 21.070 ms (48,599.0 files/s), a 1.158× speedup. With zero slack and high memory-pressure telemetry, admission falls back to CPU in 24.411 ms with an AI-QoS-exhausted deferral. This supports only maintenance-visible open-file envelope refresh, not hardware-backed credential release, a persistent KEM hierarchy, or foreground AI QoS recovery.

These measurements produce the main placement rule used by the design: AES-GCM data writes are latency-sensitive and remain CPU-first, while ML-KEM batches are elastic and may use the GPU only when producer slack and telemetry permit it. The result would be weaker if the paper reported only the faster GPU primitive; the important systems observation is the asymmetry between the data plane and the batch plane.

6.3 Correctness and negative controls

The GPU integrity test passes leaf, Merkle, and empty-tree parity cases against OpenSSL. The FUSE self-test passes journal/checkpoint/anchor unit checks and a generation-replay regression: appending an older generation after a newer mapping does not supersede the newer mapping, and decrypting a generation-2 ciphertext/tag under generation 1 fails. The retained generation fault matrix then runs the final FUSE binary through partial update, clean remount, torn journal tail, stale snapshot replay, and adversarial older-generation append cases. Its parsed journals show no duplicate generated (*block*, *generation*) pairs; every trial receives an explicit oracle verdict, with no silent-corruption or unexpected-liveness verdict. The clean-remount regression writes 128 KiB, calls `fsync`, remounts, byte-compares plaintext, and verifies that the sidecar does not contain the complete input sequence. A separate final-binary test flips the persisted HMAC-authenticated envelopes and observes `EKEYREJECTED`. These results strengthen the generation and envelope claims; the daemon matrix below supplies the selected daemon-crash evidence.

The POSIX-scope audit records the narrow interface: default encrypted-file `mmap` fails with `ENODEV`; disjoint writers recover the expected blocks; rename and directory `fsync` return `ENOTSUP`; user xattrs round-trip while internal xattrs are hidden; writeback-cache/direct-IO `mmap` capabilities are not requested; and lower-filesystem xattr atomicity and directory durability remain delegated.

Auxiliary storage/GPU diagnostics are deliberately scoped. The retained host-pinning and managed-storage probes show same-buffer CPU/GPU checksum agreement plus CUDA pointer diagnostics, but they do not prove UVM migration suppression or final FUSE data-path DMA.

The file-backed anchor produces the deliberately negative result: restoring an entire backing-directory snapshot is classified as `rollback_visible`, i.e., the previous committed baseline remains readable. The result remains a negative control rather than rollback resistance.

The TPM artifacts record anchor latency, fixed properties, PCR reads, NV public attributes for index `0x01500010`, and a transient PCR-policy probe that unseals under the current PCR digest but rejects a drifted digest. The hardware-freshness recovery matrix gives explicit oracle verdicts for stale disk/new TPM, new disk/stale TPM, missing index, changed PCRs, authorization failure, interrupted NV update, and normal replay-after-advance. The real-TPM rows

Table 7: Recovery/freshness outcomes and their interpretation.

Retained result	Interpretation	Still missing
Clean FUSE remount	Envelope/key hierarchy can recover the written file after clean unmount/remount	Crash timing across all D/J/C boundaries
Metadata HMAC tamper	Unauthenticated envelope metadata fails closed	Full ciphertext/journal tamper campaign
File-anchor replay visible	File-backed witness is a negative control	External freshness state
TPM replay-after-advance fail closed	Current provisioned NV index can reject stale disk state after anchor advance	Persistent PCR policy and lifecycle
Daemon/app fault matrix	Retained daemon and SQLite/dbm.dumb cut points recover to acceptable oracle states	Physical power loss, drive caches, and broader workloads
SQLite/dbm.dumb stale snapshots	Two workloads fail closed on TPM-backed stale replay	Broader workload matrix

cover replay, missing index, changed-PCR probe behavior, and wrong authorization; the new-disk/stale-TPM and interrupted-update rows are final-binary command-path fault injection, not physical power-loss evidence. This is replay-after-advance evidence, not a claim that the persistent filesystem anchor itself is PCR-sealed.

The broader crash/replay matrices are selected-boundary regression evidence, not complete app-level crash coverage. The daemon power-fault campaign kills the final binary at one-block data write, journal append/`fsync`, logical-size/checkpoint xattr, anchor update, `fsync`, remount-validation, and authenticated-read cut points; all nine rows retain a marker, daemon `SIGKILL`, and an acceptable oracle verdict. The same manifest ties application-level commit coverage to 20 SQLite WAL/FULL selected-boundary trials, three SQLite DELETE/EXTRA `fdatasync` app-crash trials, and a `dbm.dumb` TPM replay row; a separate audit finds no unclassified retained recovery row. This is selected daemon/app evidence, not physical power-loss, kernel-crash, drive-cache, arbitrary-workload, or multi-block raw-write atomicity certification.

Table 7 summarizes the interpretation and the next experiment required to strengthen each row.

6.4 QoS Recovery, Ablations, and Sensitivity

Retained tegrastats/CUPTI artifacts show platform pressure reaching the mounted daemon: they record 8 and 5 daemon-throttled flushes, respectively. This remains wiring evidence rather than TensorRT/AI p99 recovery or fast-notification bypass.

The app-level result uses SQLite transaction latency as the foreground workload. Table 8 is generated from the retained four-mode bundle. Mounted storage pressure raises SQLite p99 from 6.4 ms to 13.8 ms and causes one 10 ms

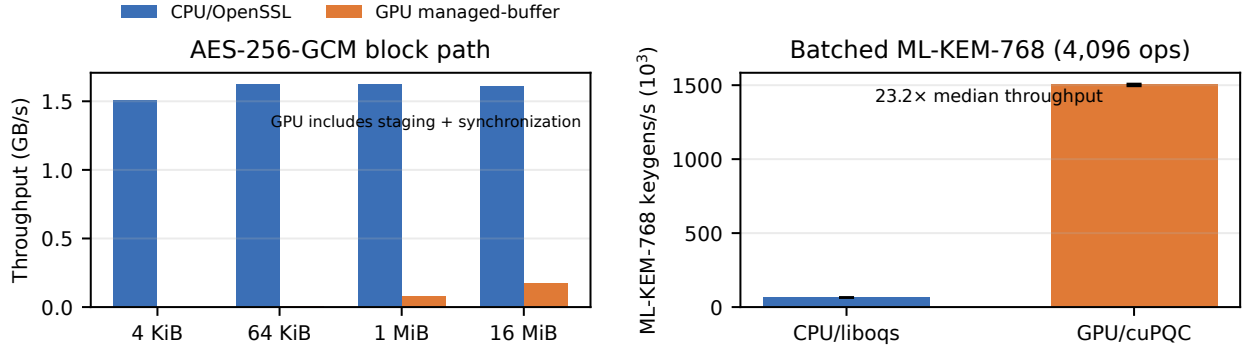


Figure 4: Placement result for RQ2/RQ5. CPU AES-GCM exceeds managed-buffer GPU for the data plane; cuPQC improves batched ML-KEM-768. Bars are medians with observed 5th–95th-percentile whiskers.

Table 8: SQLite foreground recovery under mounted secure-storage pressure; latencies are p50/p95/p99 in ms and CPU/GPU are utilization percent.

Mode	p50/p95/p99	Miss	MB/s	CPU/GPU	Samp.	Thr.
App	4.9/6.3/6.4	0	0.0	4/10	28	0
Unthrottled	6.1/9.6/13.8	1	6.8	10/10	31	0
Simple	5.3/7.0/8.2	0	2.3	4/10	28	26
AEGIS-Q	5.4/7.5/8.8	0	2.7	6/10	29	44

deadline miss. AEGIS-Q marks foreground files latency-sensitive, throttles only elastic background writes, and recovers to 8.8 ms p99 with no misses at 2.7 MB/s. The simple controller reaches 8.2 ms p99 at 2.3 MB/s. This is SQLite recovery, not general AI-inference QoS.

The sensitivity bundle varies budget, sampling interval, threshold, queue depth, and writer intensity. Its baseline p99 is 6.4 ms; 80 ms sampling and 128 KiB chunks remain at 7.0/7.2 ms, high threshold disables throttling (0 sleeps, 8.8 ms), and two writers is fragile (12.3 ms, one miss). No-throttle and no-slack fallbacks pass, and the scripted hysteresis trace records 9 transitions and 8 flips.

Together, the ablation rows attribute observed behavior to four mechanisms: the authenticated FUSE format accounts for the warm-cache filesystem cost, elastic-write throttling accounts for SQLite p99 recovery, the GPU lane helps only the mounted open-file key refresh workflow with slack, and the TPM anchor converts the file-anchor replay negative control into fail-closed replay-after-advance behavior. The sensitivity sweep then tests whether the QoS result depends on one constant rather than a stable controller regime.

7 Discussion and Limitations

7.1 Interpretation of the placement result

The useful result is asymmetry, not an aggregate “GPU speedup.” On Thor, CPU/OpenSSL fits the AES-GCM data path, while the GPU helps independent ML-KEM batches and measured envelope refresh. AEGIS-Q therefore exposes an elastic maintenance lane; current telemetry can throttle

mounted FUSE writes but still does not justify foreground AI-QoS recovery.

7.2 Security limits

The per-file envelope is authenticated and block records are AEAD-protected, but the mount password remains the root credential. The implementation has no hardware-backed credential release, audited rotation protocol, privileged-attacker defense, or side-channel evaluation. File anchors are replayable. TPM mode requires an administrator-provisioned index; retained probes cover fail-closed replay-after-advance and transient PCR-policy behavior, not persistent PCR-bound key release under provisioned TPM policy.

7.3 Evaluation limits and roadmap

The three-run primitive measurements cannot establish steady-state throughput, tail latency, energy efficiency, or portability. The frozen-contract matrix provides warm-cache plaintext, gocryptfs, and AEGIS-Q rows, but it is not yet a complete fscrypt, dm-crypt, fs-verity, dm-integrity, OP-TEE, SPDK, or GPUDirect Storage comparison. Historical fscrypt and dm-crypt fio outputs are retained only for traceability, not current comparison evidence, because they do not sweep the same request sizes, cache states, queue depths, sync modes, and application workloads.

The POSIX envelope is also narrow: ordinary read/write/fsync and narrow disjoint writers are evaluated, while shared mmap, rename, directory fsync, and arbitrary conflicting writes are not deployment claims.

The crash story has the same boundary. The repository retains a conservative crash-audit map, a daemon SIGKILL cut-point matrix, a SQLite selected-boundary oracle campaign, SQLite/dbm.dumb same-store TPM replay-after-advance runs, and SQLite syscall-exact app-crash timing, but it still lacks physical power-loss, kernel-crash, drive-cache, and evidence across broader workloads. Stronger claims still need kernel-encryption baseline rows, exact I/O/cache/clock controls, foreground p99 telemetry control, power-fail injection

with a provisioned TPM policy, a non-NVIDIA UMA platform for CUDA-independent deployment, and side-channel testing if claimed.

7.4 Deployment takeaway

The evaluated deployment envelope is a local edge service whose foreground SQLite database shares mounted secure storage with elastic background writes. This case study is the same `sqlite-mounted-qos-recovery-2026-06-27` contract used by Figure 1 and Table 8, not a separate anecdotal demo. The useful property is policy separation plus TPM replay-after-advance fail-closed behavior; it does not extend to kernel-encryption replacement, TensorRT p99 recovery, unmanaged power loss, or CUDA-independent deployment.

7.5 CUDA-specific boundary

The implementation depends on CUDA managed memory and cuPQC. It uses `cudaMemPrefetchAsync` and stream synchronization as executor-local locality and ordering operations, not storage-DMA registration or permissions. Porting the policy requires a separate proof of the target API’s allocation, synchronization, and device-I/O semantics.

8 Related Work

AEGIS-Q separates storage protection, GPU/storage placement, and PQC freshness; it is not a stronger deployed encryption layer. `fsencrypt` and `dm-crypt` are mature kernel baselines [2, 3]; `gocryptfs` is the closest user-space encrypted-filesystem comparison point [10]. `fs-verity` and `dm-integrity` validate different integrity boundaries, so this revision treats plaintext/`gocryptfs` warm-cache rows as current evidence and leaves a full `fsencrypt/dm-crypt` matrix open. AEGIS-Q’s distinct artifact is the combination of a per-file authenticated FUSE format, storage-visible latency/elastic policy, optional GPU maintenance with CPU fallback, and externally anchored replay checks in one mounted prototype.

Prior GPU/storage systems validate different assumptions. FlashNeuron and Fastensor use SSD staging for learning workloads [1, 11]; SnuQS and InfScaler motivate managed storage capacity [4, 9]; SnuQS alone is not a validation of this filesystem format [9]. GPUfs, SPDK, and GPUDirect Storage need API and hardware evidence beyond ordinary FUSE `pread/pwrite`. Edge pressure is real [12], and FlashNeuron illustrates why storage-backed execution needs separate foreground evidence [1, 12]. Batch GPU work appears under datacenter assumptions [1, 11, 13], including Fastensor/`fsencrypt-GPU` comparisons [11, 13]; `fsencrypt-GPU` itself is not a matched baseline for this prototype [13]. OP-TEE and `dm-crypt` use different protection boundaries [2, 8]. Thus the portable idea is measured admission, while CUD-A/cuPQC evidence is platform-specific.

ML-KEM establishes shared secrets rather than per-block bulk encryption [6], so AEGIS-Q keeps AES-GCM in the data plane and uses ML-KEM only for the optional batched key-plane experiment. OP-TEE provides a different trust boundary [8]; it does not make a replayable file witness

fresh. A TPM freshness root still needs explicit NV authorization, key-release policy, and recovery semantics beyond the present measurements.

9 Conclusion

AEGIS-Q is an evidence-first secure edge-storage prototype: secure-storage pressure raises foreground SQLite p99 from 6.4 ms to 13.8 ms with one deadline miss; AEGIS-Q recovers 8.8 ms, zero misses, and 2.7 MB/s background progress. It does not claim direct NVMe-to-UVM DMA, eBPF/io_uring completion bypass, GPU constant-time behavior, persistent PCR-bound freshness, power-loss certification, foreground AI p99 recovery, or portability.

References

- [1] Jonghyun Bae, Jongsung Lee, Yunho Jin, Sam Son, Shine Kim, Hakbeom Jang, Tae Jun Ham, and Jae W. Lee. 2021. FlashNeuron: SSD-Enabled Large-Batch Training of Very Deep Neural Networks. In *19th USENIX Conference on File and Storage Technologies (FAST 21)*. 387–401.
- [2] Clemens Fruhwirth. 2005. New methods in hard disk encryption: `dm-crypt` and LUKS. *Linux Journal* 2005, 138 (2005).
- [3] Michael Halcrow, Theodore Ts’o, et al. 2015. Ext4 Encryption: Design and Implementation of Linux Filesystem Encryption. *Proceedings of the Linux Symposium* (2015).
- [4] Borui Li, Tiange Xia, and Shuai Wang. 2025. InfScaler: Enabling Efficient ML Inference Serving on Multi-Accelerator Edge Devices. In *ACM/IEEE Design Automation Conference (DAC)*.
- [5] National Institute of Standards and Technology. 2015. *FIPS 180-4: Secure Hash Standard (SHS)*. Technical Report. NIST.
- [6] National Institute of Standards and Technology. 2024. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. Technical Report. NIST.
- [7] NVIDIA Corporation. 2024. *NVIDIA Jetson Orin Nano Developer Kit Platform Specifications*.
- [8] OP-TEE. 2018. OP-TEE: Open Portable Trusted Execution Environment Specification. In *GlobalPlatform Technology Conference*.
- [9] Daeyoung Park, Heehoon Kim, Jinpyo Kim, Taehyun Kim, and Jaemin Lee. 2022. SnuQS: Out-of-Core Cache Staging for Virtualized Memory Storage Systems. In *Proceedings of the International Conference on Supercomputing*.
- [10] Jakob Spennberg et al. 2017. `gocryptfs`: A Modern FUSE-based Encrypted Filesystem. In *USENIX Security Association*.
- [11] Jia Wei, Xingjun Zhang, Longxiang Wang, and Zheng Wei. 2023. Fastensor: Optimize the Tensor I/O Path from SSD to GPU for Deep Learning Training. *ACM Trans. Archit. Code Optim.* 20, 4 (2023).
- [12] Feng Zhang, Chenyang Zhang, Jiawei Guan, Qiangjun Zhou, Kuangyu Chen, Xiao Zhang, Bingsheng He, Jidong Zhai, and Xiaoyong Du. 2025. Breaking the Edge: Enabling Efficient Neural Network Inference on Integrated Edge Devices. *IEEE Transactions on Cloud Computing* (2025).
- [13] Xingjun Zhang, Longxiang Wang, et al. 2023. `fsencrypt-GPU`: GPU-Accelerated Filesystem Encryption for High-Throughput Secure Storage. In *ACM Symposium on Cloud Computing*.